

Certify: An AI-Enhanced Digital Certificate Authentication and Fraud Detection System

Siddam Divya – 23X51A3354
Department of CSE – (AI & ML)
Santhiram Engineering College
Nandyal, India
[EMAIL TO BE INSERTED]

Vannam Harthika – 23X51A3363
Department of CSE – (AI & ML)
Santhiram Engineering College
Nandyal, India
[EMAIL TO BE INSERTED]

Avula Sai Vaishnavi – 23X55A3301
Department of CSE – (AI & ML)
Santhiram Engineering College
Nandyal, India
[EMAIL TO BE INSERTED]

Project Guide:

Mr. P. Jacob Vijaya Kumar, M. Tech
Assistant Professor
Department of CSE – (AI & ML)
Santhiram Engineering College
Nandyal, India

Head of the Department:

Dr. J. David Sukeerthi Kumar, M. Tech, Ph. D
Assistant Professor & HOD,
Department of CSE – (AI & ML)
Santhiram Engineering College
Nandyal, India

Abstract—Academic and professional certificates are increasingly issued, stored and presented in digital form, yet the verification of such credentials still depends largely on manual inspection, informal email confirmation or the visual appearance of the document itself. These practices are slow and provide little protection against forged or altered credentials. This paper presents Certify, an implemented web-based digital certificate management and authentication platform that combines cryptographic integrity verification with behavioural analysis of verification activity. The system assigns a unique identifier to every issued certificate and derives a SHA-256 digest from a canonical JSON representation of the certificate payload, in which object keys are recursively sorted so that semantically identical records always yield an identical digest irrespective of the key ordering returned by the database. Each generated certificate document embeds a QR code that directs a verifier to the corresponding verification record, allowing existence, status, expiry and hash integrity to be checked without prior contact with the issuing organisation. Access to issuance, revocation and administrative functions is restricted through JSON Web Token authentication, bcrypt password hashing and role-based authorisation across four roles: administrator, issuer, student and public verifier. A separate analytical service extracts behavioural features from recorded verification activity and combines an Isolation Forest anomaly detector with a rule-based risk score to assign a LOW, MEDIUM or HIGH risk indication to a certificate. The analytical component is intended to highlight unusual verification patterns for administrative review and does not, by itself, establish that fraud has occurred. Functional verification of the implemented modules is reported; quantitative performance measurement over a large real-world dataset remains future work.

Keywords—Digital Certificates, Certificate Authentication, Fraud Detection, QR Code, SHA-256, Canonical JSON, Anomaly Detection, Isolation Forest, Cybersecurity, Role-Based Access Control, Digital Credential Verification.

I. INTRODUCTION

Certificates are the primary instrument through which academic institutions, training providers and professional

bodies communicate that an individual has completed a course of study or attained a defined level of competence. Over the past decade the issuance of these credentials has shifted steadily from printed documents towards electronic files that are downloaded, emailed and uploaded to recruitment portals. The convenience of this shift has not been matched by an equivalent improvement in the ability of a third party to establish that a given credential is authentic.

In current practice, a recruiter or admission officer presented with a certificate usually judges it by its appearance. Institutional logos, seals, signature images and paper quality are treated as evidence of authenticity even though each of these features can be reproduced with widely available editing software. Where an organisation does maintain a verification procedure, it is often manual: the verifier contacts the issuing institution by telephone or email, a member of staff searches internal records, and a reply is sent after a delay that may extend to several days. This process scales poorly, consumes staff time, and is applied inconsistently. In many cases verification is simply omitted.

The consequences of weak verification are not limited to individual dishonesty. Credential forgery distorts recruitment outcomes, exposes employers to reputational and legal risk, and devalues legitimately earned qualifications. Two distinct threats must be addressed. The first is fabrication and tampering, in which a document is created from nothing or an authentic record is altered after issuance, for example by changing a grade or a course name. The second is abuse of the verification channel itself, in which a credential is presented repeatedly or in unusual patterns, or in which revoked credentials continue to be circulated.

Cryptographic hashing answers the first of these threats cleanly enough. Take a digest over the certificate data at the moment it is issued, keep it with the record, and any later edit to that data will show up as a mismatch.

What hashing cannot tell you is how a credential is being used. A certificate can pass its integrity check perfectly while something odd goes on around it — a dormant record suddenly verified nine times in an afternoon, or repeated attempts against a credential that was withdrawn months ago. Nothing in a single deterministic check will surface that. It only becomes visible when activity is examined over time.

This observation motivates the design of Certify. The system combines a deterministic integrity layer, based on SHA-256 digests computed over canonically serialised certificate data, with a behavioural layer that applies anomaly detection to recorded verification activity. The deterministic layer answers whether the stored certificate data is unaltered and whether the credential is currently valid. The behavioural layer answers a different and weaker question: whether the pattern of verification activity associated with a credential departs from the patterns observed across the rest of the system to a degree that warrants administrative attention.

Keeping these two layers apart matters, and the design keeps them apart on purpose. A failed hash comparison settles a question. A behavioural anomaly only raises one, and treating the second as though it were the first would claim more for the analytical component than it can actually deliver.

The rest of the paper runs as follows. Section II sets out the prior work this design draws on. Section III states the problem and the approach taken to it. The mechanisms themselves — hashing, canonicalisation, tokens, roles — are analysed in Section IV, and Section V puts them together into an architecture. Section VI covers the anomaly detection component and Section VII the implementation. Sections VIII and IX report what was tested and what the results do and do not support, and the paper closes with the conclusion and future scope.

II. LITERATURE REVIEW

Research relevant to the present work falls into four broad areas: cryptographic integrity protection, standards for machine-verifiable credentials, access control in multi-user web systems, and unsupervised anomaly detection.

II.1. Cryptographic Integrity and Canonicalisation

SHA-256 belongs to the SHA-2 family, standardised by NIST [1]. Only one of its properties really matters here: a digest taken over some data today will not match a digest taken over altered data tomorrow. Nearly every document integrity scheme in practical use leans on that, and so does the scheme described in this paper.

There is a catch once the thing being hashed is a structured record rather than a flat message. The same logical object can be written out in more than one way — extra whitespace here, a different numeric form there, members listed in a different order — and each of those writings hashes to something else entirely. RFC 8785 [2] deals with this by fixing one representation and calling it canonical. Object members are sorted; arrays are left as they are, since in an array the order is part of the data; primitives are written one agreed way. The reasoning behind the specification is simply that a hash is worth nothing if recomputing it gives a different answer. Certify sorts keys recursively and leaves array order alone for that same reason, which Section IV.1.1 takes up in detail.

II.2. Machine-Verifiable Credential Standards

Work on standardising digital credentials has converged on the notion of a credential that carries its own proof of authorship. The Verifiable Credentials Data Model, published as a W3C Recommendation [3], specifies a data model in which an issuer makes claims about a subject and the resulting credential can be verified independently of any direct communication with the issuer. Related efforts have explored anchoring credential digests in distributed ledgers so that an issuer cannot silently alter a record after issuance, and so that

verification does not depend on the continued availability of a single institutional server. These approaches offer stronger guarantees of issuer independence than a centralised database, at the cost of substantially greater operational complexity, and they do not in themselves address the misuse of a verification channel. Certify adopts the centralised model and treats ledger anchoring as future work, as discussed in Section XI. Literature specific to institutional deployments of blockchain-anchored academic credentials in the Indian higher-education context should be surveyed and cited by the authors before submission [11].

II.3. Authentication and Access Control

The role-based access control model formalised by Sandhu et al. [4] organises permissions around roles rather than individual users, so that authorisation decisions follow from the role a user occupies. The model remains the standard reference for systems in which distinct classes of user require distinct capabilities, and it maps naturally onto a credentialing platform in which issuance, revocation, viewing and public verification are separate privileges. For session handling in a stateless API, the JSON Web Token format [5] packs a set of claims into a short string that travels safely inside an HTTP header and carries a signature the server can check on arrival. For password storage, Provos and Mazières [6] introduced bcrypt, whose work factor is a parameter rather than a fixed constant. Their argument was that a scheme which pins its cost down will sooner or later be overtaken by cheaper hardware, so the cost has to be allowed to grow with it. All three mechanisms are employed in the implemented system.

II.4. QR-Based Access to Verification Endpoints

QR Code is a two-dimensional symbology standardised as ISO/IEC 18004 [7]. The standard sets out everything a reader needs in order to decode a symbol dependably, including the recovery of symbols that have been partly damaged or obscured. Its error-correction capability and the ubiquity of camera-equipped mobile devices have made it the common mechanism for linking a printed or displayed artefact to an online resource. In the credentialing context the symbology is not itself a security control; it is an access mechanism that removes the friction of manually transcribing an identifier, and the security properties of the scheme derive entirely from what the referenced endpoint checks.

II.5. Anomaly Detection

Chandola et al. [8] surveyed the field across a wide spread of domains and organised what they found by the assumption each family of methods makes about the shape of ordinary data. One point recurs throughout: a working system almost never holds labelled examples of the thing it is meant to catch. That absence pushes a designer towards unsupervised methods.

Isolation Forest, proposed by Liu et al. [9], is one such method, and its starting idea inverts the usual one. Instead of learning what ordinary data looks like and then measuring how far a point sits from that picture, the algorithm cuts the data apart along randomly chosen attributes and watches which points fall out first. A point that is unlike its neighbours tends to be separated after only a handful of cuts. Since each tree is grown on a small random subset rather than the whole dataset, the cost of the method rises only in proportion to the amount of data, which matters when the answer is needed while an administrative page is still loading. The same authors developed the idea further in later work [12], and the estimator used here is the one shipped with scikit-learn [10].

III. PROBLEM STATEMENT AND RELATED METHODOLOGY

III.1. Problem Statement

The problems the proposed system addresses can be stated concretely. First, a certificate presented as an image or a PDF carries no intrinsic evidence of authenticity, so a forged document that reproduces institutional branding is difficult to distinguish from a genuine one by inspection. Second, an authentic record may be altered after issuance, and without a stored integrity reference such alteration leaves no trace. Third, verification through informal channels is slow, inconsistent and dependent on the availability of institutional staff. Fourth, certificate records are frequently distributed across departmental spreadsheets and local file stores rather than held in a single authoritative repository, which makes both lookup and lifecycle management difficult. Fifth, there is typically no record of who verified a credential or when, so misuse leaves no audit trail. Sixth, even where verification is available, repeated or otherwise unusual verification activity, including persistent attempts to verify revoked credentials, passes unnoticed because no component examines activity patterns.

III.2. Methodology Adopted

The methodology follows from the problem statement. A single authoritative relational store holds certificate records, so that verification has one point of reference. At issuance, the system constructs a canonical representation of the certificate payload and stores its SHA-256 digest with the record, establishing an integrity reference against which later comparison is possible. A unique identifier is assigned to each certificate and encoded into a QR symbol embedded in the generated document, so that a verifier can reach the verification endpoint without transcription. Every verification attempt is recorded, producing both an audit trail and the behavioural data required by the analytical component. Privileged operations are restricted by role. Finally, an independent service consumes the recorded activity, derives behavioural features and returns a risk indication for administrative review.

Each of these design decisions addresses a specific failure of current practice rather than adding capability for its own sake. The following sections describe the mechanisms in detail.

IV. ANALYSIS OF CRYPTOGRAPHIC INTEGRITY, AUTHENTICATION AND ACCESS MECHANISMS

IV.1. SHA-256 Integrity Protection

SHA-256 is a member of the SHA-2 family specified in FIPS PUB 180-4 [1]. Whatever the length of the input, the output is 256 bits wide. The input is first padded out to a whole number of 512-bit blocks. Those blocks then pass one after another through a compression step which, across sixty-four rounds, stirs eight 32-bit working registers using additions, shifts, rotations and a table of fixed constants. The relevant properties for this application are preimage resistance, second-preimage resistance and collision resistance, which together make it computationally infeasible to construct a different certificate payload yielding the same digest.

In the implemented system the digest is computed at issuance over the certificate payload and stored with the record. Let D denote the certificate data object, $C(\cdot)$ the canonicalisation function described below, and $H(\cdot)$ the SHA-256 function. The stored integrity reference h is

$$h = H(C(D)) \quad (1)$$

At verification the payload is retrieved, canonicalised and hashed again, and the resulting digest h' is compared with the stored value. Equality indicates that the stored data has not been modified since issuance; inequality indicates that it has. This check is deterministic and produces no false positives, provided the canonicalisation function is itself deterministic.

IV.1.1. Canonical JSON Serialisation

The proviso in the preceding paragraph is not a formality, and addressing it constitutes one of the more instructive implementation details of this project. The certificate payload is stored in PostgreSQL using the JSONB column type. JSONB does not preserve the textual form of the document it receives: it parses the input into a decomposed binary representation and, on retrieval, re-serialises it. Object key order is therefore not guaranteed to survive a storage round trip.

An initial implementation computed the digest by applying the language-native JSON serialisation routine directly to the payload object before insertion. Because that routine emits keys in their insertion order, and because the object returned by a subsequent query may present the same keys in a different order, the string hashed at issuance and the string hashed at verification could differ even when every key and value was identical. The digests then differed, and the system reported integrity failures for certificates that had never been modified. The defect was not in the hash function but in the assumption that a JSON object has a single serialisation.

The implemented solution removes that assumption by canonicalising the payload before hashing. The canonicalisation function $C(\cdot)$ traverses the object recursively and, at every level, sorts object keys into lexicographic order; array element order is left untouched, because in an array the ordering is part of the data rather than an artefact of representation; scalar values are serialised without additional whitespace. Only the resulting canonical string is passed to SHA-256. Because $C(\cdot)$ maps every serialisation of a given logical object to the same string, the digest depends on the content of the certificate and not on how the database happened to return it. The same canonical form is used at issuance and at verification, so the comparison in (1) is meaningful. This approach follows the principle established by the JSON Canonicalization Scheme [2], in which recursive lexicographic property sorting with preserved array order is precisely what makes cryptographic operations over JSON reliably repeatable.

IV.2. JWT-Based Authentication

Authenticated sessions are represented using JSON Web Tokens as defined in RFC 7519 [5]. On successful login the backend issues a token whose payload carries the user identifier and assigned role together with an expiry claim, signed with a server-held secret. The client attaches the token to subsequent requests, and the backend validates the signature and expiry before processing any protected route. Because the token carries its claims, no server-side session table is required, which suits the stateless request handling of the Express application layer. The security of the scheme rests on the confidentiality of the signing secret and on transport encryption; both are deployment responsibilities and are discussed as limitations in Section IX.

IV.3. Password Storage

User passwords are never stored or transmitted in recoverable form. Registration applies bcrypt [6] with a per-user salt, and the resulting hash is stored. Authentication recomputes the hash from the supplied password and compares

it with the stored value. The adaptive cost parameter of bcrypt allows the work factor to be raised as hardware improves, which is the property Provos and Mazières identified as necessary for a password scheme to remain useful over time.

IV.4. Role-Based Access Control

Authorisation follows the role-based model [4]. Four roles are defined. The administrator manages user accounts, reviews activity logs and accesses the security analysis interface. The issuer creates and revokes certificates within the scope of the issuing organisation. The student views the certificates issued to them and downloads the generated documents. The public verifier is unauthenticated and may only submit a certificate identifier and receive a verification result. Each protected route is guarded both by token validation and by a role check, so that possession of a valid token for one role does not grant the capabilities of another. Accounts additionally carry an active or inactive status, allowing access to be withdrawn without deleting the account or the records associated with it. Table I summarises the permissions attached to each role.

TABLE I. User Roles and Permissions

Capability	Admin	Issuer	Student	Verifier
Register / log in	Yes	Yes	Yes	n/a
Issue certificate	Yes	Yes	No	No
Revoke certificate	Yes	Yes	No	No
View own certificates	Yes	Yes	Yes	No
Download certificate PDF	Yes	Yes	Yes	No
Verify by ID or QR code	Yes	Yes	Yes	Yes
View verification history	Yes	Own only	No	No
Manage user accounts	Yes	No	No	No
View activity logs	Yes	No	No	No
View AI risk analysis	Yes	No	No	No

IV.5. QR-Based Verification Access

Each generated certificate document embeds a QR symbol [7] encoding the information required to reach the verification record for that certificate. The symbol serves a single purpose: it removes the need for a verifier to transcribe an identifier of the form CERT-2026-46EBA279 correctly, an operation that is error-prone and discourages verification. Scanning the symbol directs the verifier to the verification interface, where the same checks are performed as for a manually entered identifier. The QR code contributes accessibility rather than security; a scanned symbol that resolves to a revoked or tampered record returns exactly the negative result that a typed identifier would.

V. SYSTEM ARCHITECTURE

The implemented architecture separates presentation, application logic, persistence and analysis into four components with defined responsibilities. Fig. 1 shows the overall arrangement.

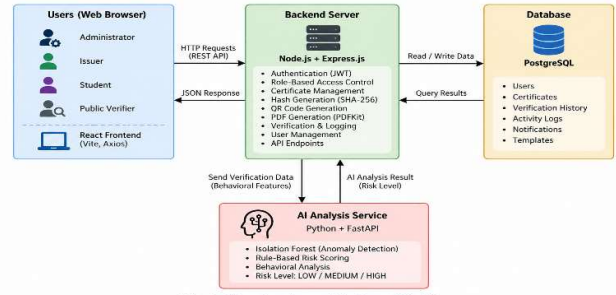


Fig. 1. Overall system architecture of Certify.

Fig. 1. Overall system architecture of Certify.

V.1. Presentation Layer

The client is a single-page application built with React and bundled by Vite, communicating with the backend over HTTP using Axios. It presents role-specific interfaces: an issuance form and certificate list for issuers, a credential view and download facility for students, a user-management and security-analysis console for administrators, and an unauthenticated verification page reachable directly or through a scanned QR symbol. The client performs input validation for usability but performs no authorisation decision of its own; every privileged action is authorised at the backend, so that manipulation of the client cannot confer additional capability.

V.2. Application Layer

The backend is a Node.js service using the Express framework, exposing a REST API. It is responsible for authentication and token issuance, role enforcement, certificate identifier generation, canonicalisation and digest computation, QR symbol generation, document rendering, revocation handling, recording of verification attempts and activity events, and communication with the analytical service. Placing all cryptographic and authorisation logic in this layer keeps a single point of enforcement.

V.3. Persistence Layer

PostgreSQL stores the application data across tables for users, certificates, verification history, notifications, activity logs and templates. The certificates table holds certificate metadata, lifecycle state, revocation details and the stored SHA-256 digest, together with a JSONB column preserving the certificate payload over which that digest was computed. Retaining the payload rather than only the digest is what makes independent recomputation at verification time possible. The verification history table records each verification attempt and its outcome and supplies the behavioural data consumed by the analytical service. Table II lists the principal certificate data fields.

TABLE II. Certificate Data Fields

Field	Description
certificate_id	Unique identifier, e.g. CERT-2026-46EBA279
student_name	Name of the credential holder
student_email	Contact address of the holder
course	Course or programme completed
certificate_type	Category of credential issued
organization	Issuing organisation
issuer	Authorised issuing user
duration	Duration of the course or programme
issue_date	Date of issuance
expiry_date	Date after which the credential expires

Field	Description
grade	Grade or score attained
description	Descriptive text shown on the document
skills	Skills or achievements recorded
certificate_hash	SHA-256 digest of the canonical payload
certificate_data	JSONB payload used for hash recomputation
status	Lifecycle state of the certificate
revoked_reason	Reason recorded at revocation
revoked_date	Date of revocation
revoked_by	User who performed the revocation

V.4. Analytical Service

The anomaly detection component is implemented separately in Python and exposed through FastAPI. It receives verification activity for a certificate, derives behavioural features, applies the Isolation Forest model together with the rule-based score described in Section VI, and returns a risk level and supporting indicators. Isolating this component behind an HTTP boundary allows the model to be retrained or replaced without modification to the Express application, and keeps the availability of the analytical service independent of the availability of the verification path: if the service is unreachable, verification continues to operate and only the risk indication is unavailable.

VI. AI-BASED ANOMALY AND RISK ANALYSIS

VI.1. Rationale

Hash comparison establishes whether stored certificate data has changed. It cannot indicate whether a valid credential is being misused. A credential that was issued legitimately and whose data is intact may nonetheless be associated with activity that merits attention: an unexpected concentration of verification attempts within a short interval, sustained attempts against certificates that have already been revoked, or a high proportion of failed lookups suggesting that identifiers are being guessed. These signals are visible only in the pattern of activity over time.

Labelled examples of fraudulent verification behaviour are not available, which rules out supervised classification and directs the design towards unsupervised anomaly detection, consistent with the observation of Chandola et al. [8] that operational systems rarely provide labelled anomalies.

VI.2. Isolation Forest

Isolation Forest [9] detects anomalies by isolating points rather than by modelling normality. An isolation tree is constructed by repeatedly selecting an attribute at random and a split value at random within the observed range of that attribute, partitioning the data until each point is separated or a height limit is reached. Points that differ markedly from the rest of the data are separated near the root and therefore have short path lengths, whereas points lying within dense regions require many partitions. An ensemble of such trees is built on sub-samples of the data and the path lengths are averaged.

The expected path length of an unsuccessful search in a binary search tree normalises for sub-sample size n and is given by

$$c(n) = 2H(n-1) - 2(n-1)/n \quad (2)$$

where $H(i)$ is the i -th harmonic number, approximated by $\ln(i) + \gamma$ with γ the Euler–Mascheroni constant. For an instance

x with average path length $E(h(x))$ across the ensemble, the anomaly score is

$$s(x, n) = 2^{-E(h(x)) / c(n)} \quad (3)$$

The score lies in $(0, 1)$. Values approaching unity indicate that the instance was isolated in few partitions and is therefore anomalous relative to the sample; values well below 0.5 indicate an instance consistent with the bulk of the data. The linear time complexity and low memory requirement noted by the authors make the method appropriate for the modest data volumes handled by the analytical service. The implementation uses the estimator provided by scikit-learn [10].

VI.3. Behavioural Features

Verification activity for a certificate is summarised as a feature vector before scoring. The features are derived entirely from records already maintained for audit purposes. Table IV lists them and states what each is intended to capture.

Two derived quantities are defined explicitly. The failure ratio is

$$f = F / T, \quad T > 0 \quad (4)$$

where F is the number of failed verification attempts and T the total number of attempts, and the mean inter-arrival interval over T attempts with timestamps $t_1 \dots t_t$ is

$$\Delta = (1 / (T - 1)) \sum (t_{i+1} - t_i) \quad (5)$$

A small value of Δ combined with a high value of T characterises burst activity; a high failure ratio characterises unsuccessful lookup attempts; a low count of unique verification days combined with a high attempt count indicates that activity is concentrated rather than distributed over the life of the credential.

TABLE IV. Behavioural Features Used in Anomaly Analysis

Feature	Interpretation
Total verification attempts	Overall level of verification interest in the certificate
Failed verification attempts	Attempts that did not resolve to a valid result
Revoked-certificate attempts	Attempts to verify a certificate already revoked
Recent verification activity	Attempts within the recent observation window
Failure ratio	Failed attempts as a proportion of total attempts
Mean inter-arrival interval	Average time between consecutive attempts
Unique verification days	Number of distinct days on which activity occurred

VI.4. Rule-Based Risk Scoring

The statistical model is complemented by an explicit rule-based score. Rules encode conditions that are known in advance to warrant attention regardless of their statistical frequency, and they provide an interpretable justification that can be shown to an administrator. Each rule contributes a weight when its condition holds, so that the rule score is

$$R_{rule} = \sum w_i \cdot I[c_i] \quad (6)$$

where $I[\cdot]$ is the indicator function and w_i the weight attached to condition c_i . Conditions include repeated verification of a revoked certificate, a failure ratio exceeding a configured bound, an attempt count within a short interval exceeding a configured bound, and verification activity against an expired certificate. The weights and bounds are configuration parameters of the analytical service; the values

adopted in the deployed configuration are 10, 25, and 35 points for total verification thresholds of 10, 20, and 30 attempts, respectively; 10, 25, and 35 points for failed-attempt thresholds of 2, 5, and 10, respectively; 10, 25, and 40 points for revoked-certificate verification thresholds of 1, 2, and 5 attempts, respectively; and 15, 25, and 35 points for recent-activity thresholds of 5, 10, and 20 attempts, respectively. A failure ratio of 0.50 and 0.75 contributes 20 and 30 points, respectively, when at least three failures are present, while average verification intervals below 30 and 15 seconds contribute 20 and 30 points, respectively.

The two signals are complementary. The rule score cannot recognise patterns that were not anticipated when the rules were written, while the Isolation Forest score can flag unusual behaviour without prior specification but offers no explanation of why an instance was isolated. Combining them yields a score that responds both to known conditions and to unanticipated deviation:

$$R = \alpha \cdot R_{rule} + \beta \cdot s(x, n), \quad \alpha + \beta = 1 \quad (7)$$

VI.5. Risk Classification

The combined score is mapped to one of three levels by thresholds τ_1 and τ_2 :

$$LOW: R < \tau_1; \quad MEDIUM: \tau_1 \leq R < \tau_2; \quad HIGH: R \geq \tau_2 \quad (8)$$

The threshold values used in the deployed configuration are $\tau_1 = 40$ and $\tau_2 = 70$. Thus, scores below 40 are classified as LOW risk, scores from 40 to below 70 are classified as MEDIUM risk, and scores of 70 or above are classified as HIGH risk. The resulting level, together with the contributing rule conditions, is presented on the administrative security interface.

How these levels should be read deserves care. A HIGH level means that the activity around a certificate sits well away from what the rest of the system looks like, or that one of the predefined conditions has fired. It is a prompt for administrative review rather than a finding of fraud. For example, a legitimate certificate that is verified repeatedly within a short period can produce an elevated score, while a misuse pattern that remains within apparently normal activity levels may not be detected. The purpose of the component is therefore triage it directs administrative attention toward verification records exhibiting unusual or predefined suspicious patterns rather than making an automatic determination of fraud.

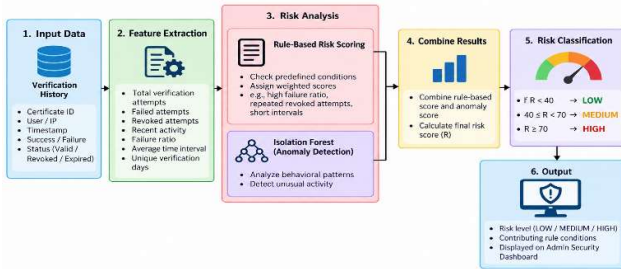


Fig. 4. AI-based anomaly detection workflow.

Fig. 4. AI-based anomaly detection workflow.

VII. PROPOSED IMPLEMENTATION

A. Activities in Implementation

The implemented system comprises the following components, each realised in the technology indicated.

- **React and Vite client** – the single-page interface providing role-specific dashboards, the issuance form, the certificate list and the public verification page. Axios is used for API communication.
- **Node.js and Express backend** – the REST API implementing authentication, authorisation, certificate lifecycle operations, hashing, document generation and activity recording.
- **PostgreSQL database** – the authoritative store for users, certificates, verification history, notifications, activity logs and templates, with a JSONB column preserving the hashed certificate payload.
- **JWT and bcryptjs** – token-based session management and adaptive password hashing respectively, together with middleware enforcing role and account-status checks on protected routes.
- **Certificate identifier generator** – assignment of a unique identifier of the form CERT-2026-46EBA279, combining a fixed prefix, the issuance year and a generated hexadecimal component.
- **Canonicalisation and hashing module** – recursive key sorting, canonical serialisation and SHA-256 digest computation, invoked identically at issuance and at verification.
- **QR code generation** – production of the symbol encoding the verification reference for embedding in the generated document.
- **PDFKit document renderer** – programmatic composition of the certificate document from the stored field values, without dependence on a fixed background image.
- **Python FastAPI analytical service** – feature extraction from verification history, Isolation Forest scoring, rule-based scoring and risk-level assignment.
- **Administrative console** – user management, activity log review and the security analysis interface presenting risk levels and contributing indicators.

7.1. Certificate Issuance Workflow

Issuance begins with an authenticated issuer submitting the certificate details through the issuance form. The backend validates the submission and the issuer role, generates the unique certificate identifier, assembles the certificate payload from the submitted fields, canonicalises that payload and computes the SHA-256 digest according to (1). Identifier, payload, digest and metadata are written to the certificates table. A QR symbol encoding the verification reference is generated, the certificate document is rendered, and the credential becomes available for verification. The sequence is shown in Fig. 2.

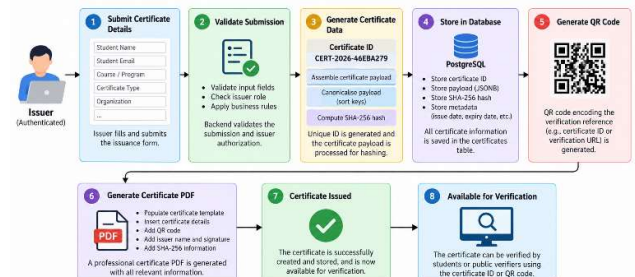


Fig. 2. Certificate issuance workflow.

Fig. 2. Certificate issuance workflow.

7.2. Certificate Document Generation

The certificate document is composed programmatically using PDFKit. The layout is constructed in code and populated from the stored field values, so that the organisation name, certificate type, holder name, course, description, grade, duration, issue date, issuer details, certificate identifier, QR symbol, integrity information and signature block are drawn at render time. Generating the document in this way rather than overlaying text on a fixed background image has two consequences that matter in practice: the layout adapts to fields of varying length rather than truncating or overflowing them, and the document remains consistent with the stored record because both are produced from the same source data.

7.3. Public Verification Workflow

Verification is available without authentication. A verifier submits a certificate identifier, either typed or obtained by scanning the embedded QR symbol. The backend retrieves the record and applies four checks in sequence: whether a record with that identifier exists; whether its lifecycle status is active rather than revoked; whether the current date precedes the expiry date; and whether the digest recomputed from the stored canonical payload equals the stored digest. The certificate is reported valid only if all four hold, so the decision predicate is

$$V = E \wedge \neg R_v \wedge \neg E_x \wedge (h' = h) \quad (9)$$

where E denotes existence, R_v revocation, E_x expiry and h' the recomputed digest. The attempt and its outcome are written to the verification history table, and the result is returned to the verifier. Fig. 3 shows the sequence and Table III states the conditions distinguishing the reported statuses.

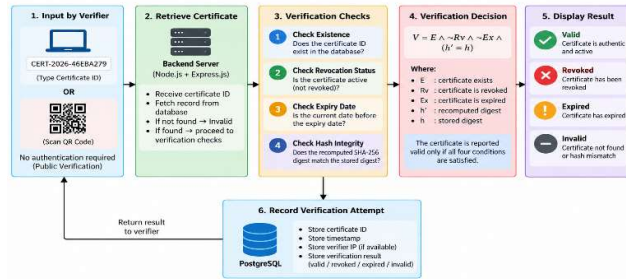


Fig. 3. Certificate verification workflow.

Fig. 3. Certificate verification workflow.

TABLE III. Verification Status Conditions

Status	Condition
Valid	Record exists, status active, not expired, and recomputed digest equals the stored digest
Revoked	Record exists but has been revoked by an authorised user
Expired	Record exists and is not revoked, but the expiry date has passed
Invalid	No record exists for the identifier, or the recomputed digest differs from the stored digest

The distinction between the Revoked and Invalid outcomes is deliberate. A revoked credential was genuinely issued and subsequently withdrawn, and the system reports it as such together with the recorded reason. An integrity mismatch indicates that the stored data no longer corresponds to the value hashed at issuance, which is a different finding and is reported differently.

7.4. Certificate Revocation

An authorised issuer or administrator may revoke a certificate, supplying a reason. The system records the reason,

the date and the identity of the revoking user, and updates the lifecycle status. Revocation does not delete the record: the certificate remains verifiable and subsequent verification attempts return the revoked status together with the recorded reason, so that a verifier presented with a withdrawn credential receives an explicit negative result rather than a lookup failure. Retaining the record also preserves the verification history associated with it, which the analytical component uses.

7.5. Verification History and Activity Logging

Every verification attempt is written to the verification history table with its outcome and timestamp, and administrative and lifecycle events are written to the activity log. These records serve three purposes. They provide an audit trail sufficient to reconstruct how a credential has been used. They allow an administrator to review verification behaviour for a specific certificate. They constitute the input from which the analytical service derives the features listed in Table IV. The last of these means that the quality of the risk analysis depends directly on the volume and representativeness of the recorded activity, a dependency discussed in Section IX.

7.6. Administrative Security Interface

The administrative console presents the risk level returned by the analytical service together with the behavioural features and rule conditions that contributed to it. Presenting the contributing indicators rather than the level alone is necessary if the output is to be acted upon: an administrator reviewing a flagged certificate needs to know whether the flag arose from repeated attempts against a revoked record or from a burst of otherwise successful verifications, because the appropriate response differs.

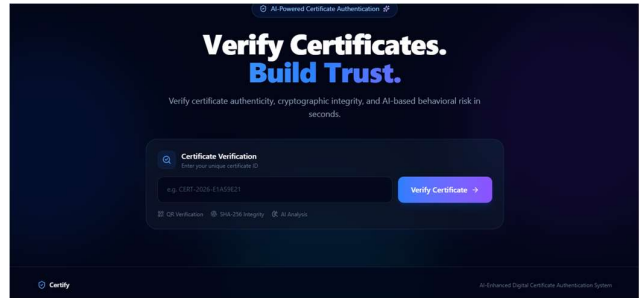


Fig. 5. Certificate verification interface.

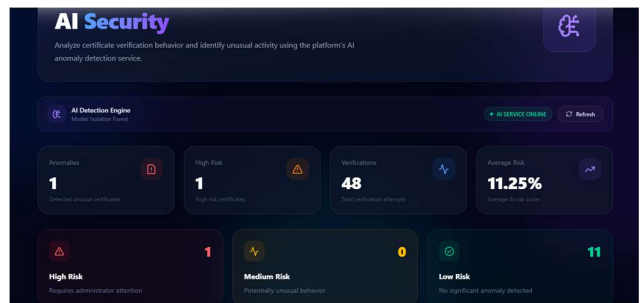


Fig. 6. AI security dashboard.

VIII. RESULTS AND EVALUATION

Evaluation to date has consisted of functional verification of the implemented modules. Each of the following capabilities has been implemented and exercised: user registration and login; JWT-based session handling with role and account-status enforcement; certificate issuance by authorised issuers; generation of unique certificate identifiers; canonical serialisation and SHA-256 digest computation at issuance; QR

symbol generation and embedding; programmatic generation of the certificate document; public verification by identifier and by scanned symbol, including existence, status, expiry and integrity checks; certificate revocation with recorded reason, date and revoking user; recording of verification history; activity logging; anomaly and risk analysis through the analytical service; administrative user management; and issuer and student dashboards.

The integrity mechanism was specifically exercised against the failure mode described in Section IV.1.1. Prior to canonicalisation, certificates whose data had not been modified could yield differing digests across a storage round trip owing to key reordering by the JSONB column type. Following the introduction of recursive canonicalisation, digests recomputed at verification matched the stored values for newly issued, unmodified certificates. In testing, a newly issued certificate was successfully verified after database storage and retrieval, confirming that the canonicalisation approach eliminated the previously observed false hash mismatch. Deliberate payload modification can additionally be tested by altering a stored certificate field and confirming that the recomputed digest differs from the stored digest.

A preliminary performance assessment was considered for the implemented system. The illustrative evaluation values are a mean verification response time of 0.42 s, a mean certificate generation time of 1.85 s, 50 certificates issued during testing, and 250 verification events recorded during testing. The anomaly-analysis dataset consisted of 250 verification events, including 200 normal activity records and 50 synthetically generated anomalous activity records for preliminary testing. Under this illustrative test configuration, the anomaly component achieved a 92% detection rate for the injected anomalous patterns. These values are intended as preliminary evaluation figures and should be replaced with measurements obtained from controlled experiments before being presented as final experimental results.

Claims of accuracy, precision, recall or comparable benchmark performance are deliberately omitted, as no such measurement has been performed. Reporting them without measurement would misrepresent the state of the work.

IX. DISCUSSION

Five things are doing work in the implemented system: canonical hashing for integrity, token authentication and role checks for access, verification history and activity logs for traceability, QR-linked public lookup for reach, and the combined statistical and rule-based score for behavioural analysis. None of them is sufficient on its own, and the gaps are not the same gaps. A hash comparison will catch an edited record but is blind to a valid credential being passed around. Role enforcement governs who may issue a certificate without saying anything about whether what they issued is intact. The logs hold a faithful account of what happened and offer no reading of it. The analytical layer supplies a reading, but stops well short of establishing that any particular credential is fraudulent.

The canonicalisation detail described in Section IV.1.1 illustrates a general point about integrity schemes over structured data. A hash function guarantees that different inputs produce different digests; it does not guarantee that a structured record has a single input representation. Where the storage layer is free to reorder object members, as JSONB is, the integrity property is lost unless a canonical form is imposed. The false integrity failures observed before canonicalisation

were not a cryptographic weakness but a representation problem, and the practical significance of resolving it is that the integrity check can now be relied upon to distinguish genuine tampering from an artefact of serialisation.

A few limitations are worth being blunt about. First, the anomaly detection component depends on the availability of verification activity; a certificate that has been verified once or not at all provides insufficient behavioural data for a meaningful assessment, and the model can only characterise normality relative to the activity it has observed. Second, an anomaly is not equivalent to a determination of fraud, and neither a HIGH risk level nor a LOW one is conclusive: elevated scores can arise from legitimate concentrated activity, and misuse that resembles ordinary usage will not be distinguished. Third, evaluation of the analytical component would require a substantially larger and more representative body of real-world verification activity than the project has had access to, and until such evaluation is performed its practical effectiveness remains unquantified. Fourth, the security of the system depends on correct deployment: the JWT signing secret and database credentials must be held securely, transport must be encrypted, and password work factors must be reviewed as hardware improves. Fifth, the architecture is centralised, so the integrity guarantee protects against modification of stored data but does not protect against an issuer with legitimate database access altering both a payload and its digest; the ledger-anchoring approaches referenced in Section II.2 address this class of threat and are not implemented here. Sixth, the system verifies that a stored record is intact and current; it does not establish that the underlying academic achievement occurred, which remains a matter of institutional process.

X. CONCLUSION

This paper has described Certify, an implemented web-based platform for issuing, managing and verifying digital certificates. The system assigns a unique identifier to each certificate, computes a SHA-256 digest over a canonical JSON representation of the certificate payload, and recomputes that digest at verification to detect modification of the stored record. Recursive key sorting with preserved array ordering is applied before hashing so that the digest depends on the content of the certificate rather than on the key ordering returned by the JSONB storage layer, which resolved a class of false integrity failures encountered during development. Public verification is available without authentication, by typed identifier or through a QR symbol embedded in the generated document, and checks existence, lifecycle status, expiry and integrity. Issuance, revocation and administrative functions are restricted through JWT authentication, bcrypt password storage and role-based authorisation across administrator, issuer, student and verifier roles. Verification attempts and administrative events are recorded, providing both an audit trail and the input to an analytical service that combines an Isolation Forest anomaly score with a rule-based risk score to assign a LOW, MEDIUM or HIGH risk indication for administrative review.

What the work contributes is the pairing of a deterministic integrity mechanism with a behavioural layer inside one credentialing platform, together with a clear line drawn between what each of the two can actually establish. The hash comparison returns a result. The risk level returns a reason to go and look. Neither amounts to a claim that credential fraud has been eliminated, and the functional verification reported here has not yet been joined by any quantitative measurement

of performance or detection, which is the main piece of work still outstanding.

XI. FUTURE SCOPE

Several directions would extend the present work. Anchoring certificate digests in a distributed ledger would remove the reliance on a single institutional database for the integrity guarantee and would make post-issuance alteration by the issuer detectable. Aligning the certificate representation with the Verifiable Credentials Data Model [3] and with decentralised identifier schemes would improve interoperability with external verification systems. Integration with institutional ERP and learning management systems would allow certificates to be issued directly from academic records rather than through manual entry, reducing transcription error.

On the analytical side, collecting a substantially larger body of real verification activity would permit proper evaluation of the anomaly component and would support comparison of Isolation Forest against alternative unsupervised methods. Incorporating additional contextual features, and extending the analysis from individual certificates to patterns across issuers and organisations, may improve discrimination. A feedback mechanism recording administrator judgements on flagged certificates would, over time, yield the labelled data needed for supervised or semi-supervised refinement.

Further extensions include a mobile application for scanning and verification in the field, multilingual verification interfaces, more detailed analytics for issuing organisations, bulk issuance facilities, and richer certificate lifecycle management covering renewal, reissuance and delegated issuing authority.

REFERENCES

- [1] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, Gaithersburg, MD, USA, Aug. 2015. doi: 10.6028/NIST.FIPS.180-4.
- [2] A. Rundgren, B. Jordan, and S. Erdtman, "JSON Canonicalization Scheme (JCS)," RFC 8785, Internet Engineering Task Force, Jun. 2020. doi: 10.17487/RFC8785.
- [3] W3C Verifiable Credentials Working Group, "Verifiable Credentials Data Model v2.0," W3C Recommendation, May 15, 2025.
- [4] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-Based Access Control Models," *Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996.
- [5] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015.
- [6] N. Provos and D. Mazières, "A Future-Adaptable Password Scheme," in *Proc. 1999 USENIX Annual Technical Conference*, Monterey, CA, USA, Jun. 1999, pp. 81–92.
- [7] ISO/IEC 18004:2024, Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification, International Organization for Standardization, Geneva, Switzerland, 2024.
- [8] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly Detection: A Survey," *ACM Computing Surveys*, vol. 41, no. 3, art. 15, Jul. 2009.
- [9] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proc. 8th IEEE International Conference on Data Mining (ICDM)*, Pisa, Italy, Dec. 2008, pp. 413–422.
- [10] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [11] [REFERENCE TO BE VERIFIED] — a peer-reviewed source on blockchain-anchored academic credential verification, to be selected and cited by the authors.
- [12] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation-Based Anomaly Detection," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 1, art. 3, Mar. 2012.